**** Inter Thread Communication (ITC)

It is a mechanism to communicate or co-ordinate between two synchronized threads within the same object to achieve a particular task.

In ITC we put a thread into wait mode by using wait() method and other thread will complete its corresponding task, after completion of the task it will call notify() method so the waiting thread will get a notification to complete its remaining task.

ITC can be implemented by the following method of Object class.

1) public final void wait() throws InterruptedException

2) public native final void notify()

3) public native final void notifyAll()

* What is the need of ITC (Inter-thread communication), The following program shows, If we don't use ITC then output is un-predictable

main thread

```
Test t1 = new Test();
t1.start();

IO.println(t1.val);
```

child thread

```
class Test extends Thread
{   int val = 0;
    for(int i=1; i<=10; i++)
          val = val + i;
}
```

val : 0 1 3 6 10 15

IQ :

--

Why wait(), notify() and notifyAll() methods are defined in Object class but not in Thread class.

wait(), notify() and notifyAll() methods are defined in Object class but not in Thread class because in order to use these methods we need SYNCHRONIZED AREA otherwise it will generate a runtime execption i.e java.lang.IllegalMonitorStateException.

In order to call these methods lock is required (due to synchronized area [current thread should be the owner]) and lock is available with Object class hence these methods are defined in Object class but not in Thread class

IQ:

What is the difference b/w sleep() and wait()

sleep()	**wait()**
1) Method of Thread class	1) Method of Object class
2) Static method	2) Non static method
3) Will not relese the object lock.	3) Will **relese the object lock.**
4) In order to call synchronized area is not compulsory	4) In order to call synchronized **area is compulsory**
5) Will automatically wake-up after completion of time period.	5) Must be notified from another thread on the same object

WAP to show that wait(), notify() and notifyAll() methods must be used from synchronized area only :

```
package com.ravi.itc;

public class ITCDemo1
{
    public static void main(String[] args)
    {
        try
        {
            Object obj = new Object();
            obj.wait();  // java.lang.IllegalMonitorStateException
        }
        catch(InterruptedException e)
        {

        }
    }
}
```

```
package com.ravi.itc;

//Program that describes, If we don't have proper communication between two
//threads then output is un-predictable

class Test extends Thread
{
    int val = 0;

    @Override
    public void run()
    {
       for(int i=1; i<=10; i++)
       {
           val = val + i;              //0    1    3   6    10    15

           try
           {
               Thread.sleep(100);
           }
           catch(InterruptedException e)
           {

           }
       }
    }
}

public class ITCDemo2
{
    public static void main(String[] args) throws InterruptedException
    {
        Test t1 = new Test();
        t1.start();

        Thread.sleep(100);

        IO.println("val value by main thread is : "+t1.val);
    }
}
```

---

public final void wait() throws InterruptedException :-

It is a predefined non static method of Object class.

It will put a thread into temporarly waiting state and it will release the Object lock, The thread will remain in the waiting state till another thread sends a notification message on the same object, After getting the notification, the waiting thread will move from WAITING to BLOCKED state and now it is waiting for Object lock to re-enter inside synchronized area. Once the lock is available, It will move to RUNNABLE State to get processor time to complete its remaining task.

It throws a checked Exception i.e InterruptedException.

public native final void notify() :-

It will wake up the single thread that is waiting on the same object.It will not release the lock , once synchronized area is completed then only lock will be automatically released.

Once a waiting thread will get the notification from the another thraed using notify()/notifyAll() method on the same object then the waiting thread will move to Blocked state and once it will get the lock then it will come to Runnable state, now It depends upon the Thread scheduler to schedule this thread for execution to complete the remaining task.

public native final void notifyAll() :-

It will wake up all the threads which are waiting on the same object.It will not release the lock , once synchronized area is completed then only lock will be released.

Step	Description	Thread State
A thread calls wait() inside a synchronized Area.	Will release the lock & goto sleep.	WAITING
Another thread calls notify() on the same object.	Wakes up one waiting thread.	WAITING to BLOCKED
Notified thread try to re-acquire the lock.	In Blocked state, waiting for Object lock.	
Lock becomes available.	Thread moves to Runnable.	
Scheduler will pick the thread.	Thread becomes Running.	

```
package com.ravi.itc;

class Demo extends Thread
{
    int val = 0;

    @Override
    public void run()
    {
        synchronized(this)
        {
        IO.println("Child thread got the lock");

        for(int i=1; i<=10; i++)
        {
            val = val + i;
        }
        IO.println("Task is completed by child thread, Sending the notification");
        notify();
        }
    }
}

public class ITCDemo3
{
    public static void main(String[] args)
    {
       IO.println("Main thread started!!!!!");
       Demo d1 = new Demo();
       d1.start();


       synchronized(d1)
       {
           IO.println("Main thread has acquired the lock");
           IO.println("Main thread is going for wait and releasing the lock");
           try
           {
            d1.wait();  //lock is released
           }
           catch(InterruptedException e)
           {
             e.printStackTrace();
           }
           IO.println("Main thread notified");
           IO.println(d1.val);

       }


    }
}
```

```
package com.ravi.itc;

class Account
{
    private double balance = 10000;

    public synchronized void withdraw(double amount)
    {
        IO.println("Going for withdraw");
        if(amount > this.balance)
        {
            try
            {
                IO.println("Low balance calling to dad for deposit the amount");
                wait();
            }
            catch(InterruptedException e)
            {
            }
        }

        this.balance -= amount;
        IO.println("After withdraw the balance is :"+this.balance);

    }

    public synchronized void deposit(double amount)
    {
        IO.println("Going for deposit");
        this.balance += amount;
        IO.println("Balance afer deposit :"+this.balance);
        notify();
    }
}

public class ITCDemo4
{
    public static void main(String[] args)
    {
       Account account = new Account();

       Thread son = new Thread()
       {
          @Override
          public void run()
          {
              account.withdraw(5000);
          }
       };
       son.start();


       Thread dad = new Thread()
       {
          @Override
          public void run()
          {
              account.deposit(10000);
          }
       };

       dad.start();
    }

}
```
